

Reviewer Pack — Moment Matrix Rank Lower Bound for Max-CUT SOS Approximation

Entry b1a344420349 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 14:54:32 UTC

Moment Matrix Rank Lower Bound for Max-CUT SOS Approximation

Entry ID: b1a344420349 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 14:54:32 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For any n -vertex graph with max-CUT value $\geq 0.878n^2$, the rank of its degree- d SOS moment matrix is $\geq \Omega(n)$ if $d < \log n / \log \log n$.

Rationale (proposer's reasoning):

The moment matrix's rank captures the algebraic complexity of polynomial certificates for max-CUT. A lower bound on this rank implies that the SOS hierarchy requires super-logarithmic depth to approximate the cut, revealing structural constraints on polynomial identities.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed1dac8fe88791ea

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_graph(n):
    graph = [[0] * n for _ in range(n)]
    edges = list(combinations(range(n), 2))
    num_edges = len(edges)
    for i in range(num_edges):
        u, v = edges[i]
        if random.random() < 0.5:
            graph[u][v] = graph[v][u] = 1
    return graph

def max_cut_polynomial(graph):
    n = len(graph)
    poly = 0
    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j]:
                poly += f"x{i} * x{j}"
    return poly

def degree_d_mononials(n, d):
    mononials = set()
    variables = list(range(n))
    def generate_monomial(monomial, depth):
        if depth == d:
            mononials.add(''.join(sorted(monomial)))
            return
        for var in variables:
            if var not in monomial:
                generate_monomial(monomial + (var,), depth + 1)
    generate_monomial((), 0)
    return sorted(mononials)

def gram_matrix(mononials, graph):
    n = len(graph)
    m = len(mononials)
    G = [[0] * m for _ in range(m)]
```

```

for i in range(m):
    for j in range(i, m):
        monomial_i = monomials[i].split('*')
        monomial_j = monomials[j].split('*')
        count = 0
        for var in set(monomial_i + monomial_j):
            if var in monomial_i and var in monomial_j:
                count += graph[int(var)][int(var)]
        G[i][j] = count
        G[j][i] = count
return G

def matrix_rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if any(matrix[j][i] != 0 for j in range(rank)):
            for j in range(i, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            for j in range(n):
                if i != j:
                    factor = -matrix[j][i] / matrix[i][i]
                    for k in range(n):
                        matrix[j][k] += factor * matrix[i][k]
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    graph = generate_random_graph(n)
    d = math.floor(math.log(n) / math.log(math.log(n)))
    poly = max_cut_polynomial(graph)
    monomials = degree_d_monomials(n, d)
    G = gram_matrix(monomials, graph)
    rank = matrix_rank(G)
    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank >= 0.1 * n
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[0]}")
else:
    print("RESULT: INCONCLUSIVE insufficient data or mapping undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6a56c87.py", line 108, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6a56c87.py", line 87, in run_trial
    poly = max_cut_polynomial(graph)
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e6a56c87.py", line 33, in max_cut_polynomial
    poly += f"x{i} * x{j}"
TypeError: unsupported operand type(s) for +=: 'int' and 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error before producing results; no data to evaluate conjecture |
next: Debug the polynomial construction code to fix the int-str TypeError

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47060
2	propose	ollama_remote	qwen3:8b	0	0	25395

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	qwen3:8b	0	0	24093
4	novelty	ollama_remote	qwen3:8b	0	0	20559
5	novelty	ollama_remote	qwen3:8b	0	0	13683
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20006
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11500
8	judge	ollama_remote	qwen3:8b	0	0	16471

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 178766 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/b1a344420349.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/b1a344420349.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/b1a344420349.tar.gz* (if generated)